

SMART LENDER

Loan Approval Prediction System

Project Documentation

Category: Finance | Domain: Machine Learning Team

Member: Shaik Dharanikota Abdul Gani

1. Introduction

ProjectTitle

SmartLender-MachineLearning-BasedLoanApprovalPredictionSystem

TeamMembers

Name	Role
Shaik Dharanikota Abdul Gani	Member
Gadiraju Gopi Sathvika	Team Lead
Madhumitha Islavath	Member
Vani Jalsutram	Member
Kanchana Katta	Member

2. ProjectOverview

Purpose

SmartLenderisamachinelearning-poweredwebapplicationdesignedtopredictthecreditworthinessof loan applicants. It enables banks and financial institutions to make faster, data-driven loan approval decisions by evaluating applicant details through trained classification models, reducing manual effort and improving consistency in credit assessment.

Features

- Predicts loan approval likelihood using applicant data such as gender, marital status, education, employment status, income, loan amount, loan term, credit history, and property area.
- Evaluates and compares four classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost — to identify the best-performing model.
- Integrates the best model (XGBoost, achieving 94.7% training accuracy and 81.1% testing accuracy) into a Flask web application for real-time predictions.
- Provides a simple, user-friendly web interface for applicants and credit officers to submit details and instantly receive a prediction.
- Designed for deployment on IBM Cloud, making the solution scalable and accessible.
- Helps reduce non-performing assets (NPAs) by supporting faster and more consistent credit risk evaluation.

Use-Case Scenarios

Scenario 1 – Fast-Track Approval for Low-Risk Applicants: A bank credit officer enters the details of a salaried applicant with a good credit history and stable income. The model predicts loan approval with high confidence, allowing the officer to fast-track the application without manual review.

Scenario 2 – High-Risk Applicant Detection: An applicant with irregular self-employment income and no credit history submits a loan request. The system flags the application as high-risk, prompting further scrutiny or document verification before proceeding.

Scenario 3 – Batch Evaluation for High-Volume Periods: A financial analyst uses the platform to evaluate multiple applicants during a high-volume period. Relying on the XGBoost model's predictions significantly reduces evaluation time while maintaining accuracy and compliance.

3. Architecture

Frontend

The frontend is built using HTML, CSS, and Jinja2 templates rendered by Flask. It provides a form-based interface where users enter applicant details (income, credit history, loan amount, etc.) and view the predicted loan approval outcome on the results page.

Backend

The backend is developed in Python using the Flask web framework. It handles incoming form data, performs any necessary preprocessing (encoding categorical values, scaling), loads the trained XGBoost model, generates a prediction, and returns the result to the frontend.

Machine Learning Pipeline

- Data collection and cleaning of the loan applicant dataset.
- Exploratory data analysis using Pandas, Matplotlib, and Seaborn.
- Feature encoding and preprocessing using Pandas and Scikit-learn.
- Model training and evaluation using Decision Tree, Random Forest, KNN, and XGBoost.
- Selection of the best-performing model (XGBoost) based on training and testing accuracy.
- Serialization of the final model (e.g., using pickle/joblib) for integration with the Flask application.

DataStorage

The application does not use a relational or NoSQL database. Applicant data submitted through the form is processed in memory for prediction; the trained model and any referenced datasets are stored as flatfiles (CSV for data, a serialized model file for the trained XGBoost model) within the project directory.

Deployment

The application is designed for deployment on IBM Cloud, allowing it to be accessed as a hosted web service.

4. SetupInstructions

HardwareRequirements

Component	Requirement
Processor	Intel i3 or above
RAM	4GB (8GB recommended)
Storage	10GB free space
System Type	64-bit OS
Display	1366×768 or higher

SoftwareRequirements

- Operating System: Windows 10/11, Linux, or macOS
- Anaconda Navigator – for Python environment management
- Python 3.8+ – core programming language
- Code Editor: PyCharm or VS Code
- Web Browser: Google Chrome or Microsoft Edge (to run the Flask app)

Required Python Packages/Skills

- NumPy
- Pandas
- Scikit-learn
- Matplotlib
- SciPy
- Seaborn
- Flask

InstallationSteps

```
# 1. Clone the repository
gitclone<GITHUB_REPO_LINK>
cd smart-lender
#2.Createandactivateavirtualenvironment(Anacondaexample) conda create
-n smartlender python=3.9
condaactivatesmartlender

#3.Installdependencies pipinstall-
requirements.txt

#4.RuntheFlaskapplication python app.py
```

```
#5.Opentheappinyourbrowser
http://127.0.0.1:5000/
```

5. FolderStructure

TheprojectfollowsasimpleFlaskapplicationstructure:

```
smart-lender/
├── app.py                # Flaskapplicationentrypoint
├── model/
│   ├── training.ipynb    # Modeltraining&evaluationnotebook
│   └── loan_model.pkl     # Serializedbest-performingmodel(XGBoost)
├── data/
│   └── loan_dataset.csv  # Applicantdatasetusedfortraining
├── templates/
│   ├── index.html        # Applicantinputform
│   └── result.html       # Predictionresultpage
├── static/
│   ├── css/
│   │   └── style.css
│   └── js/
│       └── script.js
├── requirements.txt      # Pythondependencies
└── README.md
```

KeyComponents

app.py: Defines Flask routes, loads the trained model, and handles prediction requests.

model/: Contains the training notebook/scripts and the final serialized model file.

data/: Contains the dataset used to train and evaluate the classification models.

templates/: HTML pages rendered by Flask (input form and results page).

static/:CSSandJavaScriptassetsforstylingand interactivity.

6. Running the Application

Once dependencies are installed, the application can be started with a single command:

```
pythonapp.py
```

By default, Flask runs the development server at `http://127.0.0.1:5000/`. Open this URL in a browser to access the Smart Lender interface, fill in the applicant details form, and submit it to view the loan approval prediction.

7. API/Route Documentation

The Flask application exposes the following routes:

Route	Method	Description
/	GET	Render the home page with the loan applicant input form.

Route	Method	Description
/predict	POST	Accepts form data (income, credit history, loan amount, etc.), runs the trained XGBoost model, and returns the prediction (Approved / Not Approved).

Example Request Parameters (/predict)

- Gender
- MaritalStatus
- Education
- EmploymentStatus(Self-Employed/Salaried)
- ApplicantIncome
- LoanAmount
- LoanAmountTerm
- CreditHistory
- PropertyArea

Example Response

```
{
  "prediction": "LoanApproved", "confidence":
  "High" }
```

8. Authentication

The current version of Smart Lender is designed as an internal decision-support tool and does not implement user authentication or session management. All users accessing the deployed URL can submit applicant details and receive predictions. For production deployment on IBM Cloud, it is recommended to add role-based authentication (e.g., for credit officers vs. analysts) and secure the /predict endpoint accordingly.

9. User Interface

The user interface consists of two main screens: an input form where the applicant's or officer's details are entered, and a result page that displays the predicted outcome (loan approved or not approved).

Screenshots of these screens should be added here once the demo link is available. [Insert screenshot of the input form here]

[Insert screenshot of the prediction result page here]

10. Testing

Model Evaluation

Each of the four classification algorithms — Decision Tree, Random Forest, KNN, and XGBoost — was trained on the applicant dataset using a train-test split, and evaluated using accuracy as the primary metric.

Model	Training Accuracy	Testing Accuracy
Decision Tree	—	—
Random Forest	—	—
K-Nearest Neighbors (KNN)	—	—
XGBoost (selected)	94.7%	81.1%

Note: Fill in the training/testing accuracy for Decision Tree, Random Forest, and KNN from your evaluation results. XGBoost was selected as the final model based on its superior testing accuracy.

Application Testing

- Manual form-input testing to verify correct handling of valid and invalid applicant data.
- Verification that the /predict route correctly loads the serialized model and returns predictions.
- Cross-checking sample predictions against expected outcomes for known high-risk and low-risk applicant profiles.

11. Screenshots or Demo

Demo Link:

https://drive.google.com/file/d/1VArRnDBx2UCeB8Qeh9HdLHcbemh6Mali/view?usp=drive_link

GitHub Repository: <https://github.com/YASH-P7/SMART-LENDER>

12. Known Issues

- No authentication/authorization is currently implemented.
- Model accuracy for Decision Tree, Random Forest, and KNN has not yet been documented in this report.
- The application currently supports single-applicant predictions only; batch (bulk CSV) prediction is not yet implemented.
- Limited input validation on the frontend form.

13. Future Enhancements

- Add batch/bulk predictions support for evaluating multiple applicants at once (CSV upload).
- Implement user authentication and role-based access (credit officer, analyst, admin).
- Add a dashboard with visual analytics on approval/rejection trends.
- Retrain and update the model periodically with new applicant data to maintain accuracy.
- Complete deployment on IBM Cloud with CI/CD for automated updates.
- Add explainability (e.g., SHAP values) so credit officers can see which factors influenced a prediction.